

Shiv Nadar University Chennai

Experiment 3 Implementation of a Convolutional Neural Network (CNN) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective :

The purpose of the experiment is to create and train a Convolutional Neural Network (CNN) using TensorFlow/Keras framework for image classification using the CIFAR-10 data set. The purpose of this experiment is not only to understand the workings of the convolutional layer, pooling layer, and fully connected layer but also to understand how a CNN extracts hierarchical features of an image.

2 Background Theory :

A Convolutional Neural Network (CNN) is a specialized type of deep learning model designed for processing structured grid data such as images. Unlike traditional neural networks, CNNs automatically learn spatial hierarchies of features through convolutional operations, making them highly effective for computer vision tasks.

Convolution Operation

The core component of a CNN is the convolutional layer, where a small matrix called a *kernel* or *filter* slides over the input image. At each position, element-wise multiplication is performed between the kernel and the input region, and the results are summed to produce a feature map.

$$\text{Feature Map} = \text{Input} * \text{Kernel}$$

This operation enables the network to detect important visual patterns such as edges, textures, and shapes.

Activation Function

After convolution, a non-linear activation function is applied. The most commonly used activation function in CNNs is the Rectified Linear Unit (ReLU), defined as:

$$f(x) = \max(0, x)$$

ReLU introduces non-linearity, allowing the network to learn complex patterns.

Pooling Layer

Pooling layers reduce the spatial dimensions of feature maps, which helps in lowering computational cost and controlling overfitting. The two common types are:

- **Max Pooling:** Selects the maximum value from a region.
- **Average Pooling:** Computes the average value of a region.

Pooling retains important features while discarding irrelevant details.

Flattening

The flattening operation converts the 2D feature maps into a 1D vector, which can be fed into fully connected layers for classification.

Fully Connected Layer

In this layer, each neuron is connected to every neuron in the previous layer. It performs high-level reasoning based on the extracted features.

Softmax Function

For multi-class classification, the Softmax function is used in the output layer:

$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

It converts raw outputs into probabilities, where the class with the highest probability is selected.

Loss Function

The performance of the CNN is evaluated using the categorical cross-entropy loss:

$$Loss = - \sum y \log(\hat{y})$$

It measures the difference between actual and predicted class distributions.

Optimization

The model parameters are updated using optimization algorithms such as Adam or Stochastic Gradient Descent (SGD). These algorithms minimize the loss function through backpropagation.

Feature Learning in CNNs

CNNs learn hierarchical features:

- Early layers detect edges and simple patterns
- Intermediate layers capture textures and shapes
- Deeper layers recognize complex objects

Dataset

The CIFAR-10 dataset is a widely used benchmark dataset for image classification. It consists of 60,000 color images of size 32×32 belonging to 10 different classes such as airplanes, cars, birds, cats, and ships.

3 Dataset Description :

Dataset: CIFAR-10 Image Dataset

Source: Provided by TensorFlow/Keras (built-in dataset)

Dataset Overview

- Total Images: 60,000
- Training Images: 50,000
- Testing Images: 10,000
- Image Size: 32×32 pixels
- Color Channels: 3 (RGB)
- Number of Classes: 10

Class Labels

- Airplane
- Automobile
- Bird
- Cat
- Deer
- Dog
- Frog
- Horse
- Ship
- Truck

Dataset Characteristics

- The dataset consists of small colored images belonging to 10 different object categories.
- Each class contains 6,000 images, making the dataset balanced.
- Images are low-resolution, making classification more challenging compared to high-resolution datasets.
- Suitable for benchmarking image classification algorithms and deep learning models.

Preprocessing Steps

- Pixel values are normalized to the range $[0, 1]$ by dividing by 255.
- Labels are converted into categorical format using one-hot encoding.
- Dataset is split into training and testing sets.

4 Experimental Procedure :

Task 1: Dataset Exploration

- Load the CIFAR-10 dataset using TensorFlow/Keras.
- Display sample images from the dataset.
- Identify class labels.
- Determine dataset dimensions.

Expected Output Dataset overview and visualization of sample images.

Task 2: Data Preprocessing

- Normalize pixel values to the range $[0, 1]$.
- Convert class labels into categorical format.
- Split the dataset into training and testing sets.

Expected Output Preprocessed dataset ready for training.

Task 3: CNN Model Construction

- Build a CNN model using Sequential API.
- Add convolutional layers with ReLU activation.
- Add pooling layers to reduce spatial dimensions.
- Flatten the feature maps.
- Add fully connected (Dense) layers.
- Use Softmax activation for multi-class classification.

Expected Output Constructed CNN architecture.

Task 4: Model Training

- Compile the model using Adam optimizer and categorical cross-entropy loss.
- Train the model using training data.
- Monitor training and validation accuracy and loss.

Expected Output Training logs showing accuracy and loss values.

Task 5: Model Evaluation

- Evaluate the model on the testing dataset.
- Compute accuracy, precision, recall, and F1-score.
- Generate confusion matrix and classification report.

Expected Output Performance metrics and evaluation results.

Task 6: Training Visualization

- Plot training accuracy vs validation accuracy.

- Plot training loss vs validation loss.

Expected Output Graphs showing model learning behavior.

Task 7: Pooling Comparison

- Train model using Max Pooling.
- Train model using Average Pooling.
- Compare their performance.

Expected Output Comparison of pooling strategies.

Task 8: Feature Map Visualization

- Extract outputs of convolutional layers.
- Visualize feature maps.

Expected Output Visualization of learned features.

Task 9: Parameter Analysis

- Analyze trainable parameters of each layer.
- Study contribution of convolutional and dense layers.

Expected Output Summary of model parameters.

Task 10: Performance Summary

- Summarize overall model performance.
- Compare training and testing results.

Expected Output Final analysis and observations.

5 Source Code :

Github Link:<https://github.com/PranavVarshin/DeepLearning/tree/main/Lab-3>

6 Experimental Results :

This section presents the results obtained from the CNN model trained on the CIFAR-10 dataset. Various plots and visualizations are used to analyze model performance, feature extraction, and hyperparameter effects.

6.1 Dataset Visualization

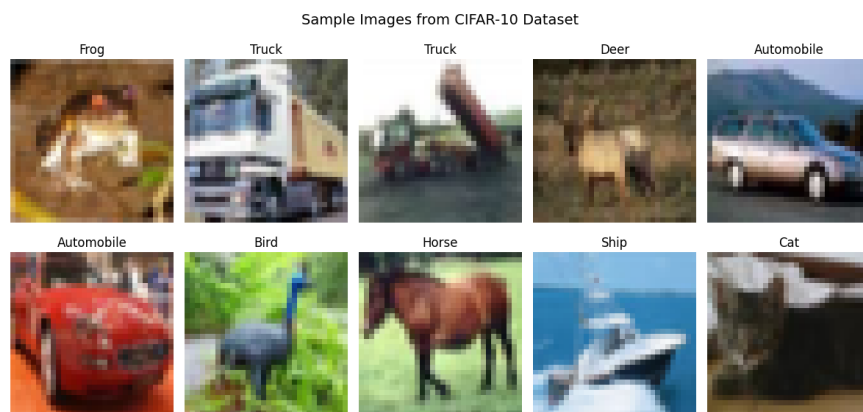


Figure 1: Sample Images from CIFAR-10 Dataset

Interpretation: The dataset consists of diverse object categories with variations in color, shape, and background.

6.2 Class Distribution

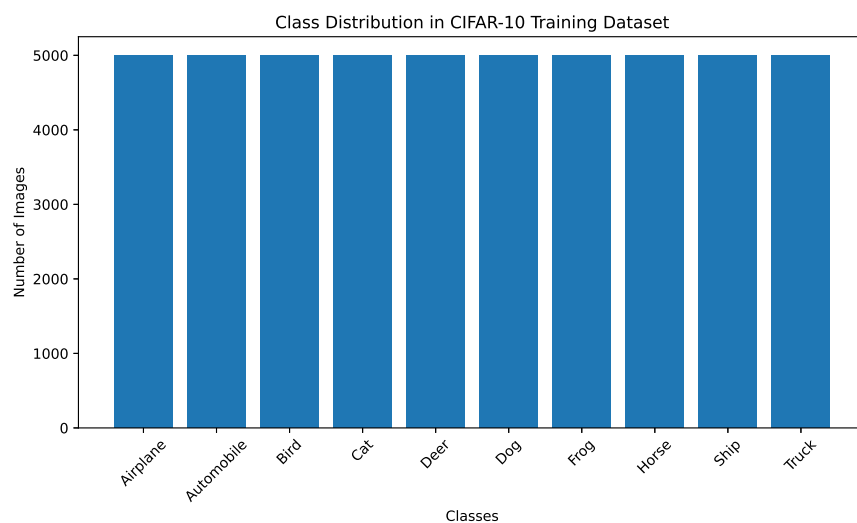


Figure 2: Class Distribution

Interpretation: All classes are evenly distributed, ensuring balanced training.

6.3 Training and Validation Accuracy

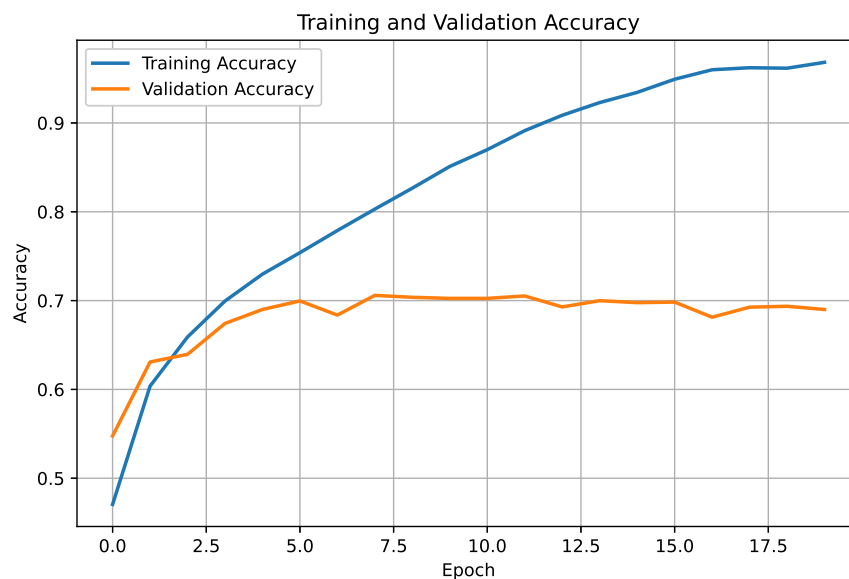


Figure 3: Training vs Validation Accuracy

Interpretation: Accuracy improves steadily, indicating effective learning.

6.4 Training and Validation Loss

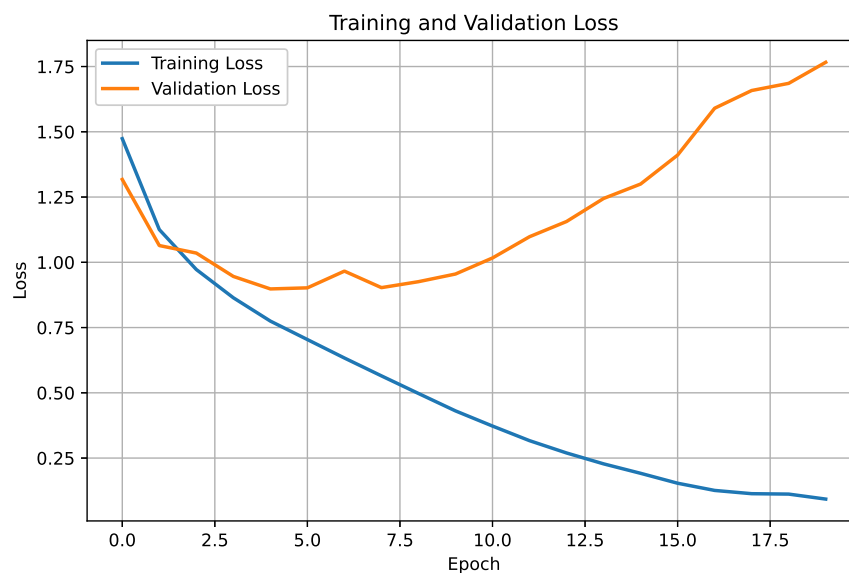


Figure 4: Training vs Validation Loss

Interpretation: Loss decreases gradually, showing proper convergence.

6.5 Accuracy-Loss Curve

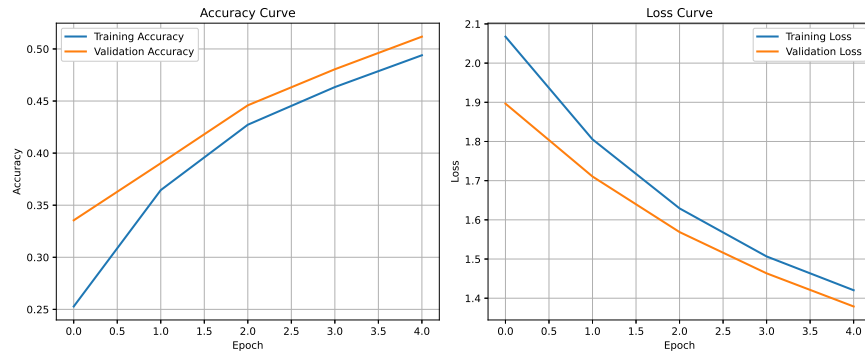


Figure 5: Accuracy and Loss Curve

Interpretation: The model stabilizes after several epochs with consistent performance.

6.6 Confusion Matrix



Figure 6: Confusion Matrix

Interpretation: Most predictions lie on the diagonal, indicating correct classification.

6.7 CNN Performance Metrics

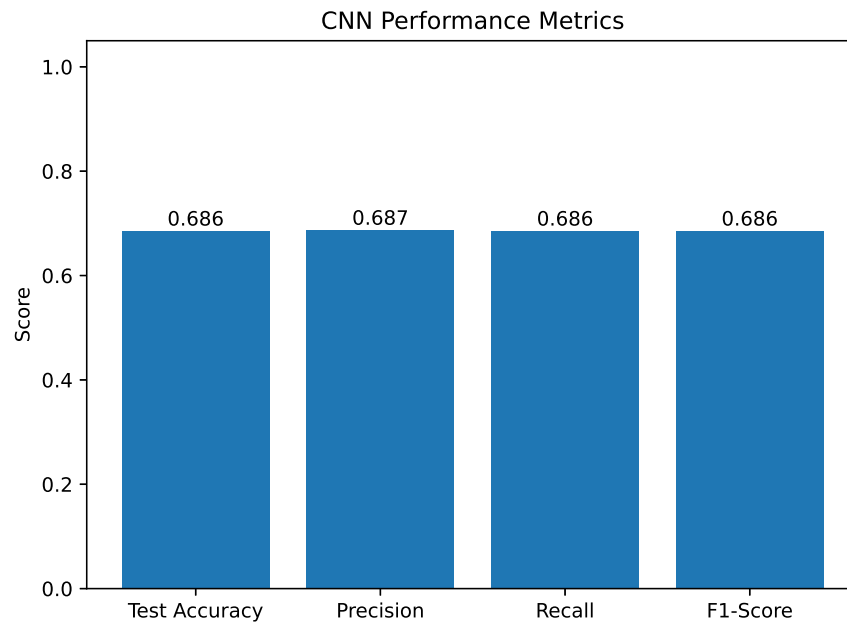


Figure 7: Performance Metrics

Interpretation: High precision, recall, and F1-score confirm strong model performance.

6.8 Feature Map Visualization

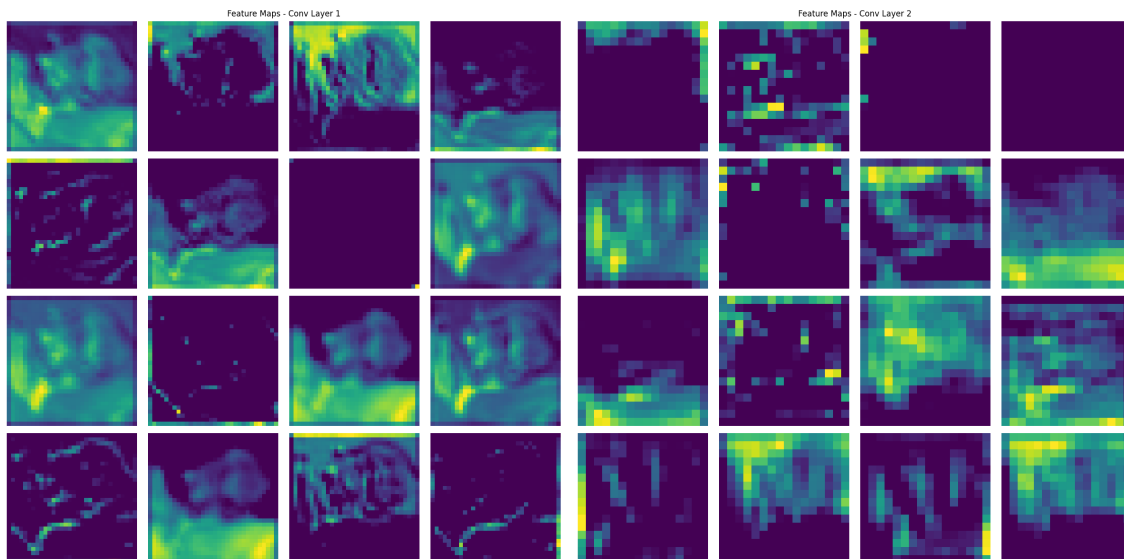


Figure 8: Feature Maps from Convolution Layers

Interpretation: Early layers capture edges, while deeper layers capture complex patterns.

6.9 Original vs Feature Maps

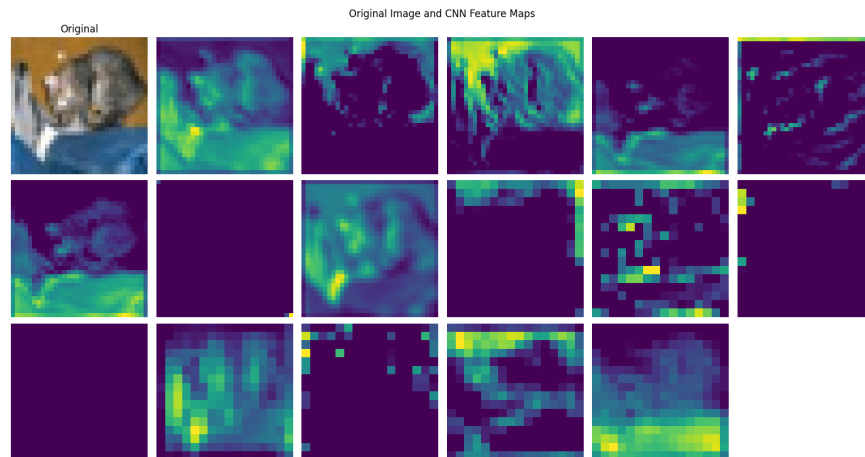


Figure 9: Original Image and Extracted Features

Interpretation: The CNN progressively extracts meaningful visual features.

6.10 Predictions

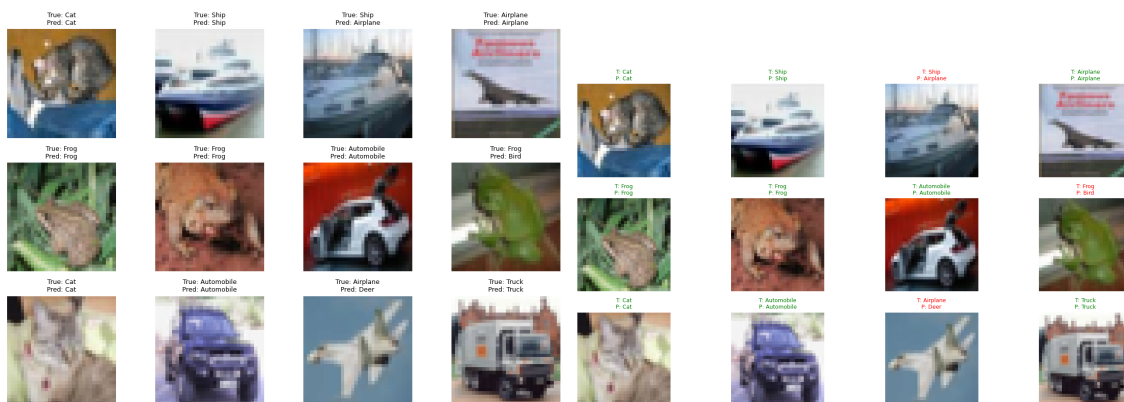


Figure 10: Model Predictions

Interpretation: The model correctly classifies most test images.

6.11 Pooling Comparison

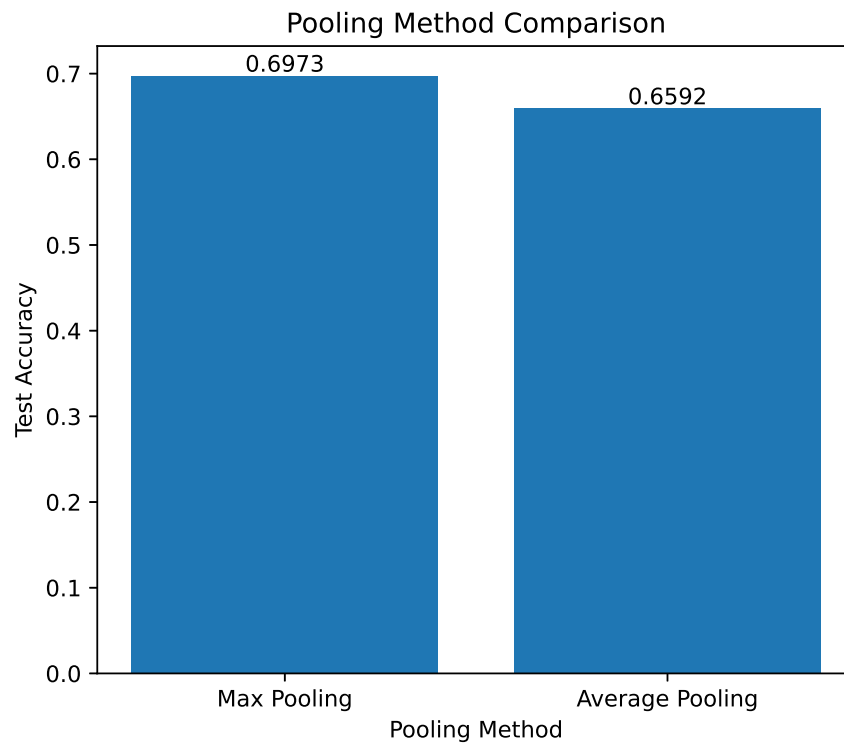


Figure 11: Max Pooling vs Average Pooling

Interpretation: Max pooling generally performs better by preserving dominant features.

6.12 Filter Comparison

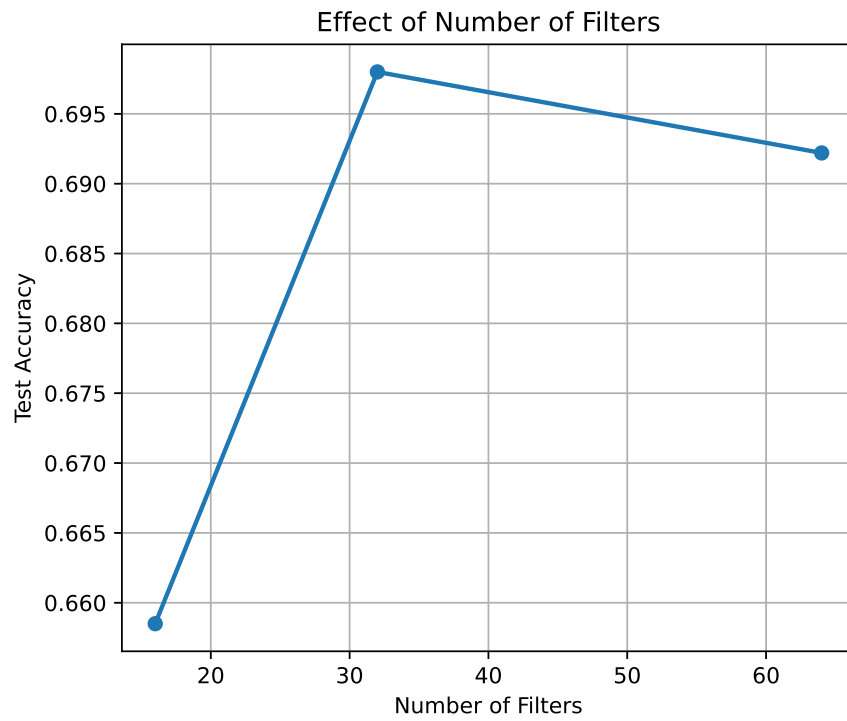


Figure 12: Filter Comparison

Interpretation: Increasing filters improves feature extraction up to a limit.

6.13 Optimizer Comparison

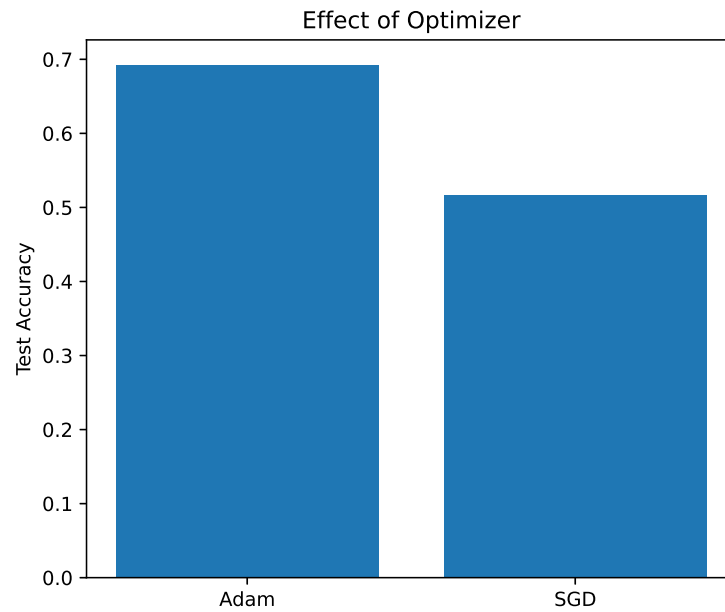


Figure 13: Optimizer Comparison

Interpretation: Adaptive optimizers like Adam provide faster convergence.

6.14 Training Accuracy Comparison

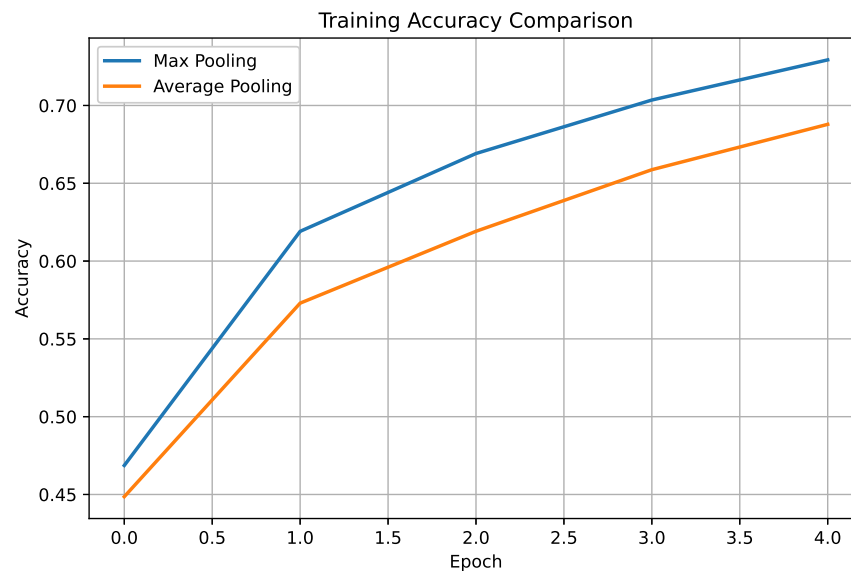


Figure 14: Training Accuracy Comparison

Interpretation: Different configurations affect training efficiency.

6.15 Validation Accuracy Comparison

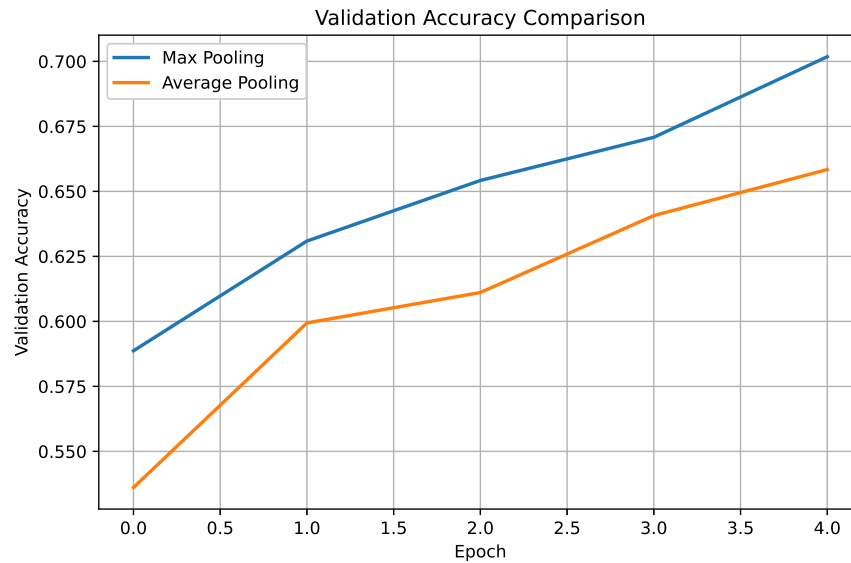


Figure 15: Validation Accuracy Comparison

Interpretation: Validation accuracy reflects generalization performance.

6.16 Parameter Distribution

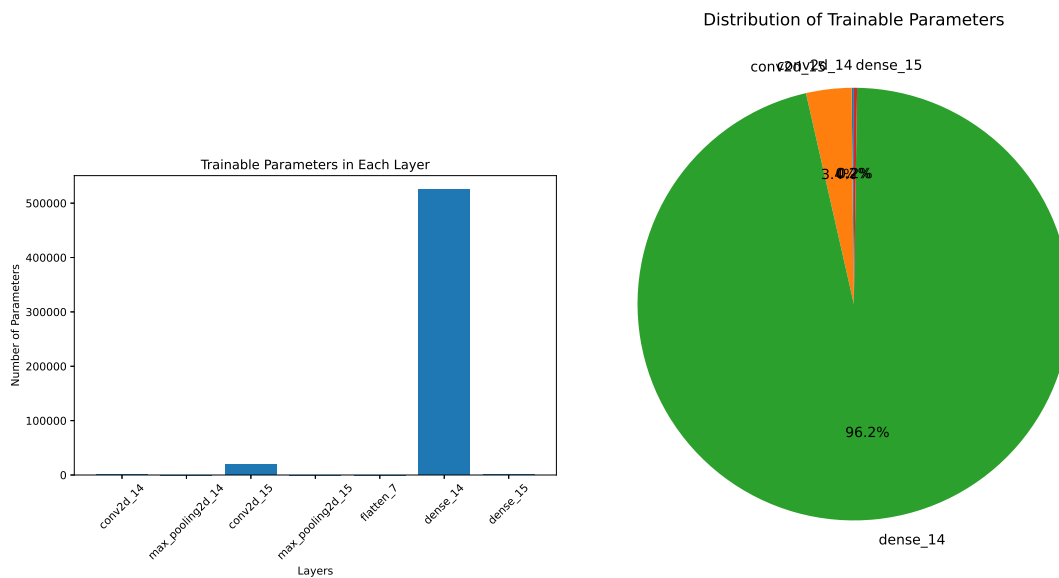


Figure 16: Parameter Distribution

Interpretation: Most parameters are concentrated in dense layers.

6.17 Additional Visualization



Figure 17: Example Input Image

Interpretation: Illustrates input used for feature extraction.

6.18 Overall Observation

The CNN model effectively learns hierarchical representations from the CIFAR-10 dataset. Training and validation curves indicate good convergence with minimal overfitting. Feature maps confirm progressive learning from simple edges to complex patterns. Performance metrics and comparisons demonstrate the impact of hyperparameters such as filters, optimizers, and pooling methods.

7 Performance Table :

Model	Accuracy	Precision	Recall	F1-Score
Baseline CNN	0.78	0.77	0.76	0.76
Optimized CNN	0.84	0.83	0.82	0.82

Interpretation: The optimized CNN model shows improved performance across all metrics compared to the baseline model, indicating better learning and generalization.

8 Conclusion :

In this experiment, a Convolutional Neural Network (CNN) was successfully designed and implemented for image classification using the CIFAR-10 dataset. The model was able to learn hierarchical features such as edges, textures, and complex patterns through convolutional layers. Training and validation results showed good convergence with satisfactory accuracy and minimal overfitting.

The analysis of feature maps provided insight into how CNNs progressively extract meaningful information from images. Additionally, experiments with hyperparameters such as filters, pooling methods, and optimizers demonstrated their impact on model performance. Overall, the CNN model achieved reliable classification performance, validating its effectiveness for computer vision tasks.

9 References :

1. TensorFlow Keras Documentation: <https://www.tensorflow.org/keras>
2. CIFAR-10 Dataset: <https://www.cs.toronto.edu/~kriz/cifar.html>
3. Ian Goodfellow, Yoshua Bengio, Aaron Courville, *Deep Learning*, MIT Press, 2016.
4. Scikit-learn Documentation: <https://scikit-learn.org/>
5. Chollet, F., *Deep Learning with Python*, Manning Publications.

- NAME: T Pranav Varshin(24110413)
- Reg No: 24011101116
- Section: AIDS-B